

Step-by-Step Project Setup Guide

High-Performance Spring Boot Microservices

Prerequisites Check

✔ Verify Your Current Setup:

```
# Check Docker Desktop
docker --version
docker-compose --version

# Check Kubernetes
kubectl version --client
kubectl cluster-info

# Check kubectl can access your cluster
kubectl get nodes

# Check Java
java --version # Should be Java 21

# Check Maven
mvn --version
```

Required Tools:

- ✔ Docker Desktop with Kubernetes enabled
- ✔ Java 21 (for Virtual Threads support)
- ✔ Maven 3.8+
- ✔ kubectl (already installed)
- ✔ Your favorite IDE (IntelliJ IDEA recommended)

Step 1: Create Project Structure

Create the main project directory:

```
mkdir high-performance-microservices
cd high-performance-microservices
```

Create the complete directory structure:

```

# Core services
mkdir -p api-gateway/src/main/java/com/example/gateway
mkdir -p api-gateway/src/main/resources
mkdir -p product-service/src/main/java/com/example/product
mkdir -p product-service/src/main/resources
mkdir -p order-service/src/main/java/com/example/order
mkdir -p order-service/src/main/resources
mkdir -p payment-service/src/main/java/com/example/payment
mkdir -p payment-service/src/main/resources
mkdir -p notification-service/src/main/java/com/example/notification
mkdir -p notification-service/src/main/resources

# Shared components
mkdir -p shared/proto-defs
mkdir -p shared/avro-schemas
mkdir -p shared/common-lib/src/main/java/com/example/common

# Infrastructure
mkdir -p infrastructure/docker
mkdir -p infrastructure/k8s/services
mkdir -p infrastructure/k8s/infrastructure
mkdir -p infrastructure/monitoring/grafana/dashboards
mkdir -p infrastructure/monitoring/grafana/datasources
mkdir -p infrastructure/monitoring/prometheus

# Scripts and documentation
mkdir -p scripts
mkdir -p docs

echo "👉 Project structure created!"

```

Step 2: Infrastructure Setup (Docker Compose)

Create the infrastructure Docker Compose file:

```

# Create docker-compose.yml in infrastructure/docker/
cd infrastructure/docker

```

infrastructure/docker/docker-compose.yml

```

version: '3.8'

services:
  # Message Broker
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.1
    hostname: zookeeper
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

```

```

    ZOOKEEPER_TICK_TIME: 2000
volumes:
  - zookeeper-data:/var/lib/zookeeper/data
networks:
  - microservices

kafka:
  image: confluentinc/cp-kafka:7.6.1
  hostname: kafka
  container_name: kafka
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
  volumes:
    - kafka-data:/var/lib/kafka/data
  networks:
    - microservices

# Databases
postgres:
  image: postgres:16-alpine
  container_name: postgres
  restart: unless-stopped
  ports:
    - "5432:5432"
  environment:
    POSTGRES_DB: microservices
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
  volumes:
    - postgres-data:/var/lib/postgresql/data
    - ./init-db.sql:/docker-entrypoint-initdb.d/init-db.sql
  networks:
    - microservices

redis:
  image: redis:7-alpine
  container_name: redis
  restart: unless-stopped
  ports:
    - "6379:6379"
  command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
  volumes:
    - redis-data:/data
  networks:
    - microservices

```

```

# Service Discovery
eureka:
  image: steeltoeoss/eureka-server:3.1.3
  container_name: eureka
  ports:
    - "8761:8761"
  networks:
    - microservices

# Monitoring Stack
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  restart: unless-stopped
  ports:
    - "9090:9090"
  volumes:
    - ../monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus-data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--web.enable-lifecycle'
  networks:
    - microservices

grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    GF_SECURITY_ADMIN_PASSWORD: admin
  volumes:
    - grafana-data:/var/lib/grafana
    - ../monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards
    - ../monitoring/grafana/datasources:/etc/grafana/provisioning/datasources
  networks:
    - microservices

jaeger:
  image: jaegertracing/all-in-one:latest
  container_name: jaeger
  restart: unless-stopped
  ports:
    - "16686:16686"
    - "14268:14268"
  environment:
    COLLECTOR_OTLP_ENABLED: true
  networks:
    - microservices

volumes:
  postgres-data:

```

```
redis-data:
kafka-data:
zookeeper-data:
prometheus-data:
grafana-data:

networks:
  microservices:
    driver: bridge
```

Create database initialization script:

```
-- infrastructure/docker/init-db.sql
-- Create databases for each service
CREATE DATABASE IF NOT EXISTS products;
CREATE DATABASE IF NOT EXISTS orders;
CREATE DATABASE IF NOT EXISTS payments;
CREATE DATABASE IF NOT EXISTS notifications;

-- Create tables for Product Service
\c products;
CREATE TABLE IF NOT EXISTS products (
  id VARCHAR(255) PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  price DECIMAL(10,2) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create tables for Order Service
\c orders;
CREATE TABLE IF NOT EXISTS orders (
  id VARCHAR(255) PRIMARY KEY,
  customer_id VARCHAR(255) NOT NULL,
  product_id VARCHAR(255) NOT NULL,
  quantity INTEGER NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  status VARCHAR(50) DEFAULT 'CREATED',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 3: Monitoring Configuration

infrastructure/monitoring/prometheus/prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
```

```
- job_name: 'spring-boot-services'
  metrics_path: '/actuator/prometheus'
  static_configs:
    - targets:
      - 'host.docker.internal:8081' # product-service
      - 'host.docker.internal:8082' # order-service
      - 'host.docker.internal:8083' # payment-service
      - 'host.docker.internal:8084' # notification-service
      - 'host.docker.internal:8080' # api-gateway
  scrape_interval: 10s
```

infrastructure/monitoring/grafana/datasources/prometheus.yml

```
apiVersion: 1
datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true
```

Step 4: Start Infrastructure

Start all infrastructure services:

```
# Navigate to docker directory
cd infrastructure/docker

# Start all services
docker-compose up -d

# Verify all services are running
docker-compose ps

# Check logs if needed
docker-compose logs -f
```

Verify services are accessible:

- Eureka: <http://localhost:8761>
- Prometheus: <http://localhost:9090>
- Grafana: <http://localhost:3000> (admin/admin)
- Jaeger: <http://localhost:16686>

Step 5: Create Parent POM

high-performance-microservices/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>high-performance-microservices</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>pom</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.1</version>
    <relativePath/>
  </parent>

  <properties>
    <java.version>21</java.version>
    <spring-cloud.version>2023.0.0</spring-cloud.version>
    <grpc.version>3.0.0.RELEASE</grpc.version>
    <resilience4j.version>2.2.0</resilience4j.version>
  </properties>

  <modules>
    <module>api-gateway</module>
    <module>product-service</module>
    <module>order-service</module>
    <module>payment-service</module>
    <module>notification-service</module>
    <module>shared/common-lib</module>
  </modules>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>${spring-cloud.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
      <dependency>
        <groupId>net.devh</groupId>
        <artifactId>grpc-server-spring-boot-starter</artifactId>
        <version>${grpc.version}</version>
      </dependency>
      <dependency>
        <groupId>io.github.resilience4j</groupId>
        <artifactId>resilience4j-spring-boot3</artifactId>
```

```
        <version>${resilience4j.version}</version>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

Step 6: Development Order

Recommended development sequence:

Phase 1: Basic Services (Week 1)

1. **Shared Common Library** - Common models, utilities
2. **API Gateway** - Basic routing and service discovery
3. **Product Service** - REST + basic GraphQL endpoints

Phase 2: Core Functionality (Week 2)

4. **Order Service** - gRPC implementation with Virtual Threads
5. **Notification Service** - Kafka consumers and WebFlux

Phase 3: Advanced Features (Week 3)

6. **Payment Service** - gRPC with GraalVM Native compilation
7. **Inter-service communication** - gRPC between services
8. **Event-driven architecture** - Kafka event flows

Phase 4: Production Ready (Week 4)

9. **Observability** - Complete monitoring, tracing, metrics
10. **Security** - OAuth2/JWT implementation
11. **Kubernetes deployment** - Production-ready manifests
12. **Load testing** - Performance validation

Step 7: Build Scripts

scripts/build-all.sh

```
#!/bin/bash
set -e

echo "🚧 Building High-Performance Microservices..."

# Build parent project
echo "🚧 Building parent project..."
mvn clean install -DskipTests

# Build each service
services=("api-gateway" "product-service" "order-service" "payment-service" "notification-service")

for service in "${services[@]}; do
    echo "🚧 Building $service..."
    cd $service
    mvn clean package spring-boot:build-image -DskipTests
    cd ..
done

echo "✅ Build completed successfully!"
```

scripts/start-dev.sh

```
#!/bin/bash
set -e

echo "🚧 Starting Development Environment..."

# Start infrastructure
echo "🚧 Starting infrastructure services..."
cd infrastructure/docker
docker-compose up -d
cd ../..

# Wait for services to be ready
echo "🚧 Waiting for infrastructure to be ready..."
sleep 30

# Start services in order
echo "🚧 Starting microservices..."
echo "Starting API Gateway..."
cd api-gateway && mvn spring-boot:run &
sleep 10

echo "Starting Product Service..."
cd ../product-service && mvn spring-boot:run &
sleep 10

echo "Starting Order Service..."
cd ../order-service && mvn spring-boot:run &
```

```
sleep 10

echo "Starting Notification Service..."
cd ../notification-service && mvn spring-boot:run &

echo "✔ All services started!"
echo "📍 Access points:"
echo "  - API Gateway: http://localhost:8080"
echo "  - Eureka Dashboard: http://localhost:8761"
echo "  - Grafana: http://localhost:3000"
echo "  - Prometheus: http://localhost:9090"
echo "  - Jaeger: http://localhost:16686"
```

Make scripts executable:

```
chmod +x scripts/*.sh
```

Step 8: Next Steps

Immediate actions:

1. ✔ Run infrastructure: `cd infrastructure/docker && docker-compose up -d`
2. ✔ Verify all infrastructure services are running
3. ✔ Create the first service (API Gateway) with the code I provided earlier
4. ✔ Test connectivity between services
5. ✔ Set up your IDE with the project structure

This Week's Goals:

- Get infrastructure running
- Build API Gateway service
- Create basic Product Service with GraphQL
- Test service discovery with Eureka

Would you like me to provide the complete code for the API Gateway first, or help you with any specific step?